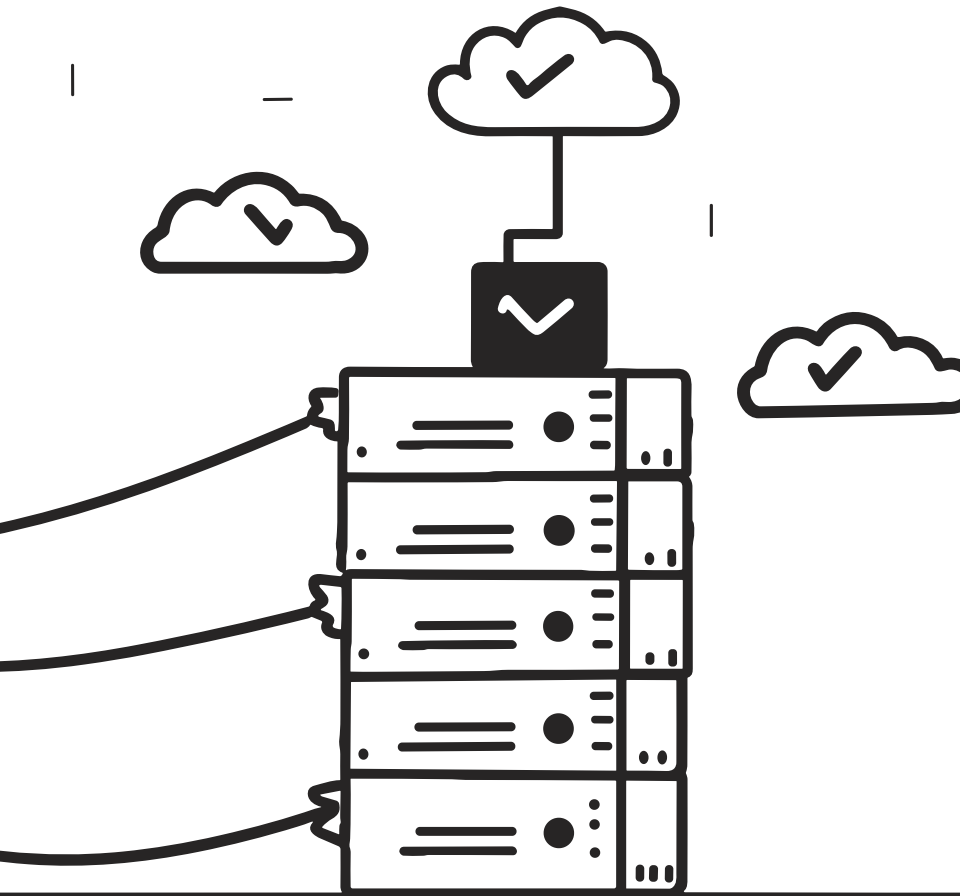




Teoria R1 — Stack, servidor i entorn executable

Presentació teòrica de transició per preparar la decisió de stack i el muntatge de l'entorn executable. Conceptes clau: llenguatge, runtime, servidor web, contenidor i comprovació mínima.

CICLE FORMATIU · R1

Índex de la sessió

01

Les peces de l'entorn

Llenguatge, runtime, servidor,
dependències i variables

02

Mapa de stacks possibles

Opcions raonables per al curs i el
moment adequat

03

Avantatges i riscos

Com justificar una decisió tècnica

04

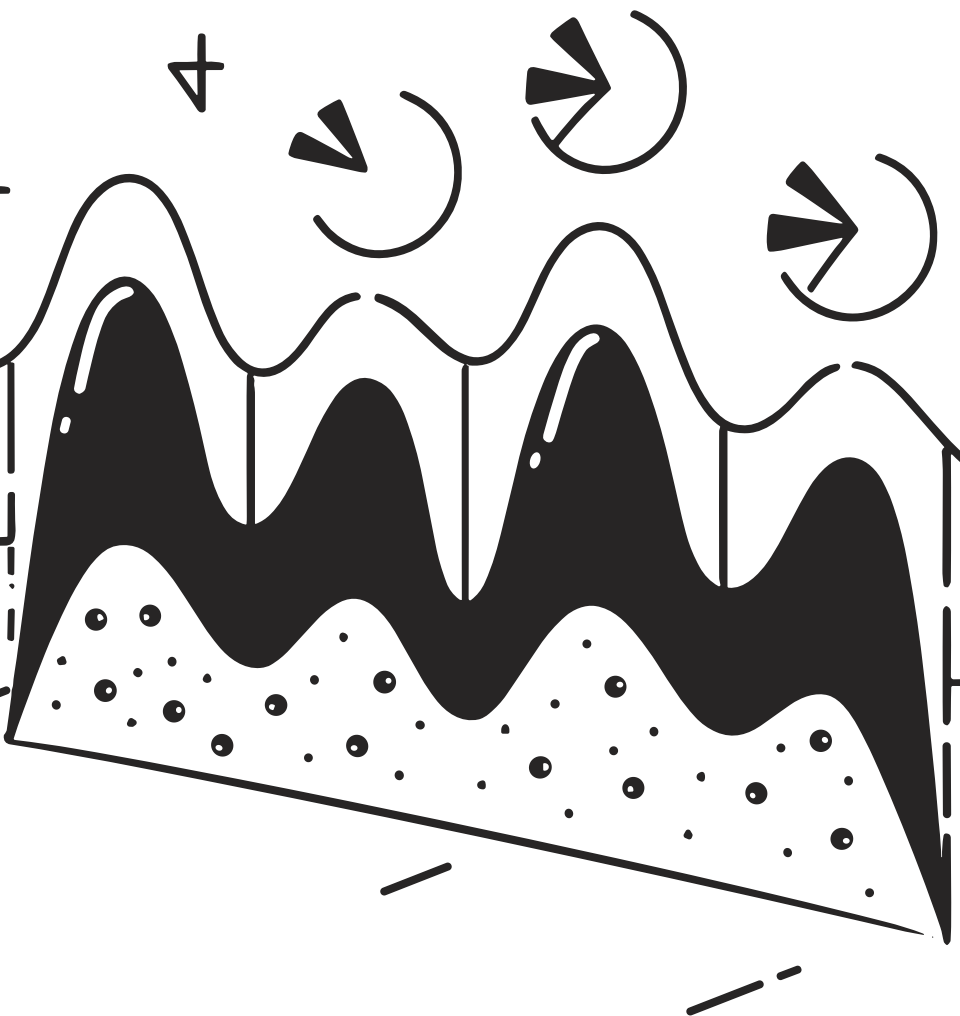
Comprovar que funciona

Ruta mínima, resposta i log

05

Docker i errors habituals

Reproduïbilitat i README d'arrencada



Les peces de l'entorn

Un entorn executable no és una sola cosa. Té capes ben diferenciades que cal conèixer per separar-les i entendre el paper de cadascuna.

Mapa de peces: de quin element parlem?



Llenguatge

Sintaxi i regles per escriure el codi. PHP, JavaScript, Python...



Runtime

Motor que executa el codi: Node.js, l'interpret de PHP, CPython...



Servidor web

Escolta peticions HTTP i les respon: Apache, Nginx, el servidor integrat de Node...



Framework

Conjunt de convencions i eines que organitzen el codi: Laravel, NestJS, FastAPI...



Dependències

Biblioteques externes que el projecte necessita. Gestionades amb Composer, npm, pip...

Decidir stack no és triar marca



La pregunta no és **"quina tecnologia és millor?"** sinó **"quina permet construir i defensar el producte ara?"**

Ara

Quin problema ha de resoldre el repte en aquest moment?

Més endavant

Què quedarà per a reptes posteriors sense bloquejar el progrés?

Mapa de stacks possibles

Opcions raonables per al curs, amb el moment natural d'ús a cada repete.

Stack	Ús raonable en el curs	Moment natural
PHP base	Entendre servidor, formularis, estat i autenticació mínima sense ocultar capes.	R1–R2
PHP + Laravel	Producte complet: rutes, controladors, vistes, BBDD, auth i API amb ecosistema productiu.	R3–R5
PHP + Symfony	Arquitectura explícita, components reutilitzables i mantenibilitat professional.	R3–R5
Node.js + NestJS	Backend TypeScript amb API, capes, DTO i contractes clars.	R3–R5
Python + FastAPI	API-first amb documentació automàtica si el projecte ho justifica.	Opció avançada

Avantatges i riscos per stack

Stack	Avantatges	Riscos o punts de vigilància
PHP base	Permet veure què passa en cada petició. Ajuda a entendre el model client-servidor.	Pot acabar amb codi desorganitzat si no es posen límits clars.
Laravel	Estructura, convencions, autenticació, migracions i proves.	Pot ocultar conceptes bàsics si s'usa abans d'entendre el flux HTTP.
Symfony	Fa visibles les decisions d'arquitectura i afavoreix dissenys robustos.	Corba d'entrada alta; pot ser pesat per a un prototip curt.
NestJS	Coherent si es domina TypeScript i es vol una API modular.	Requereix controlar moltes eines abans d'arribar al producte.
FastAPI	Ràpid per a APIs i molt clar en contractes de dades.	Pot allunyar-se del focus PHP del mòdul sense justificació docent.

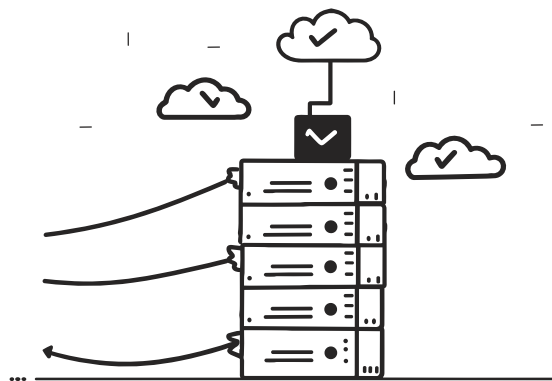
Com decidir el stack

Cinc preguntes per justificar una decisió tècnica de manera defensable.

- 1** Quin problema ha de resoldre el repte ara?
- 2** Quina part del stack genera una evidència observable?
- 3** Què sabrà explicar l'equip en una defensa curta?
- 4** Què quedarà més fàcil de provar, desplegar o revisar?
- 5** Quin risc tècnic introdueix i com es controlarà?

Comprovar que l'entorn funciona

Un entorn executable és **part de l'evidència**, no una tasca administrativa. La comprovació mínima és senzilla però obligatòria.



Ruta mínima

Defineix `/` o `/health` que retorne una resposta simple.

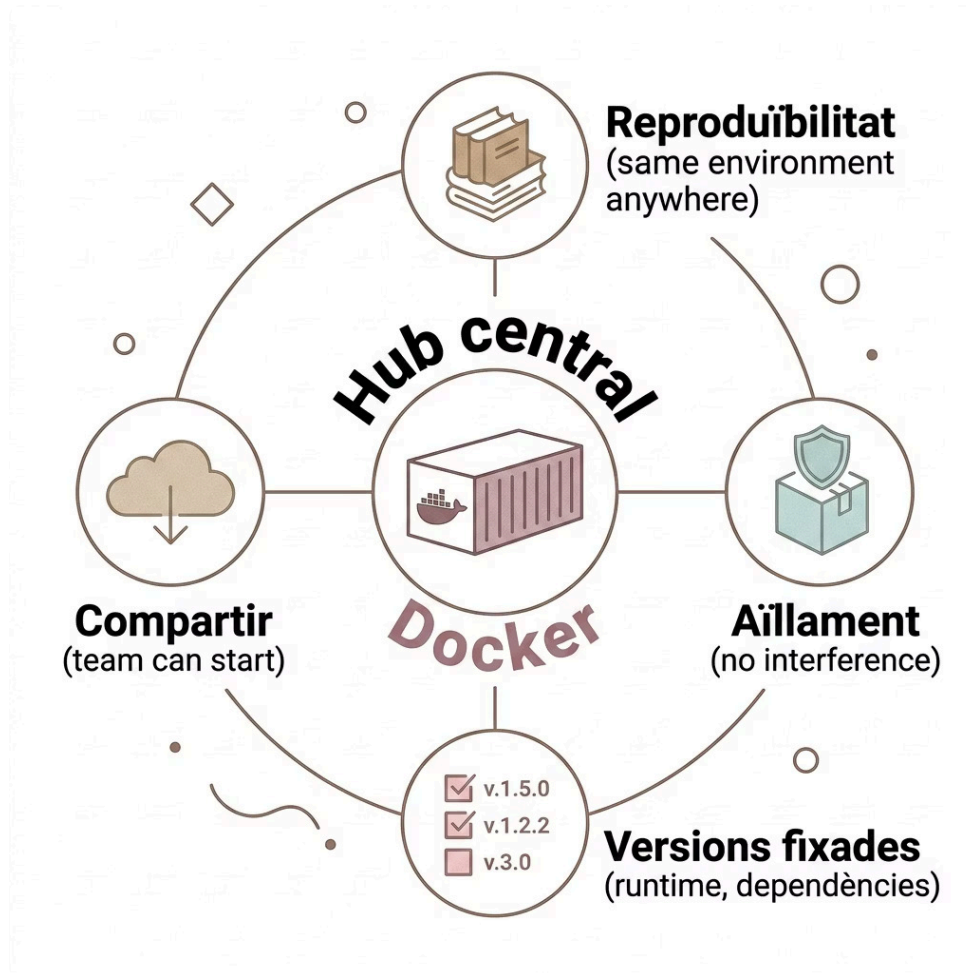
Resposta del servidor

HTTP 200 OK amb cos mínim:
`{"status":"ok"}`

Log o captura

Evidència visible de la petició i la resposta al terminal o navegador.

Docker o entorn equivalent



Què resol Docker?

- Elimina el problema "*funciona a casa meva*"
- Fixa la versió del runtime i les dependències
- Permet que qualsevol membre de l'equip arrenque el projecte

Què no resol?

- No substitueix entendre els conceptes subjacents
- No garanteix que el codi sigui correcte ni ben estructurat

Exemple: arrencada amb Docker Compose

Una comanda única que arrenque l'entorn complet. Documenta-la al README.

```
# Clonar el repositori
git clone https://github.com/equip/projecte.git
cd projecte

# Copiar variables d'entorn
cp .env.example .env

# Arrencar l'entorn
docker compose up --build

# Comprovar que funciona
curl http://localhost:8000/health
# → {"status":"ok","version":"1.0.0"}
```

❗ Les variables de versió i runtime han de quedar explícites al `docker-compose.yml` o al `.env.example` perquè el projecte no depengui d'un entorn inesperat.

EI README d'arrencada

EI README ha d'explicar com repetir l'arrencada en una altra màquina, **sense coneixement previ del projecte**.

Ha d'incloure obligatòriament:

- Versió del runtime i del llenguatge
- Comanda d'instal·lació de dependències
- Comanda d'arrencada del servidor
- URL o ruta de comprovació
- Variables d'entorn necessàries (amb `.env.example`)

Requisites

- PHP 8.2 o Node.js 20
- Docker Desktop 4.x

Arrencada ràpida

`cp .env.example .env`

`docker compose up --build`

Comprovació

GET `http://localhost:8000/health`

→ `{"status":"ok"}`

Variables d'entorn

`APP_PORT=8000`

`DB_HOST=db`

`DB_NAME=projecte`



Errors habituals

"Funciona a casa meva"

La versió del runtime no està documentada i difereix entre màquines.
Solució: fixa la versió al `docker-compose.yml` o al README.

README incomplet

Falta la comanda d'arrencada, la URL de comprovació o les variables d'entorn. Una altra persona no pot reproduir l'entorn sense preguntar.

Versions no documentades

El projecte depèn d'una versió concreta de PHP o Node que no apareix en cap fitxer. Quan canvia la màquina, l'entorn es trenca.

Preguntes de comprovació

Respon aquestes preguntes per saber si has assolit els conceptes de la sessió.

Peces

Quina diferència hi ha entre llenguatge, runtime, servidor web i framework?

Entorn

Què ha de passar perquè puguis dir que l'entorn és executable?

Comprovació

Quina comanda o URL usaràs per comprovar que el projecte arranca?

README

Què ha d'explicar el README perquè una altra persona pugui repetir l'arrencada?

Docker

Quin problema resol Docker o l'entorn equivalent en aquest projecte?

Stack

Quin avantatge i quin risc té el stack que proposes per al repte?